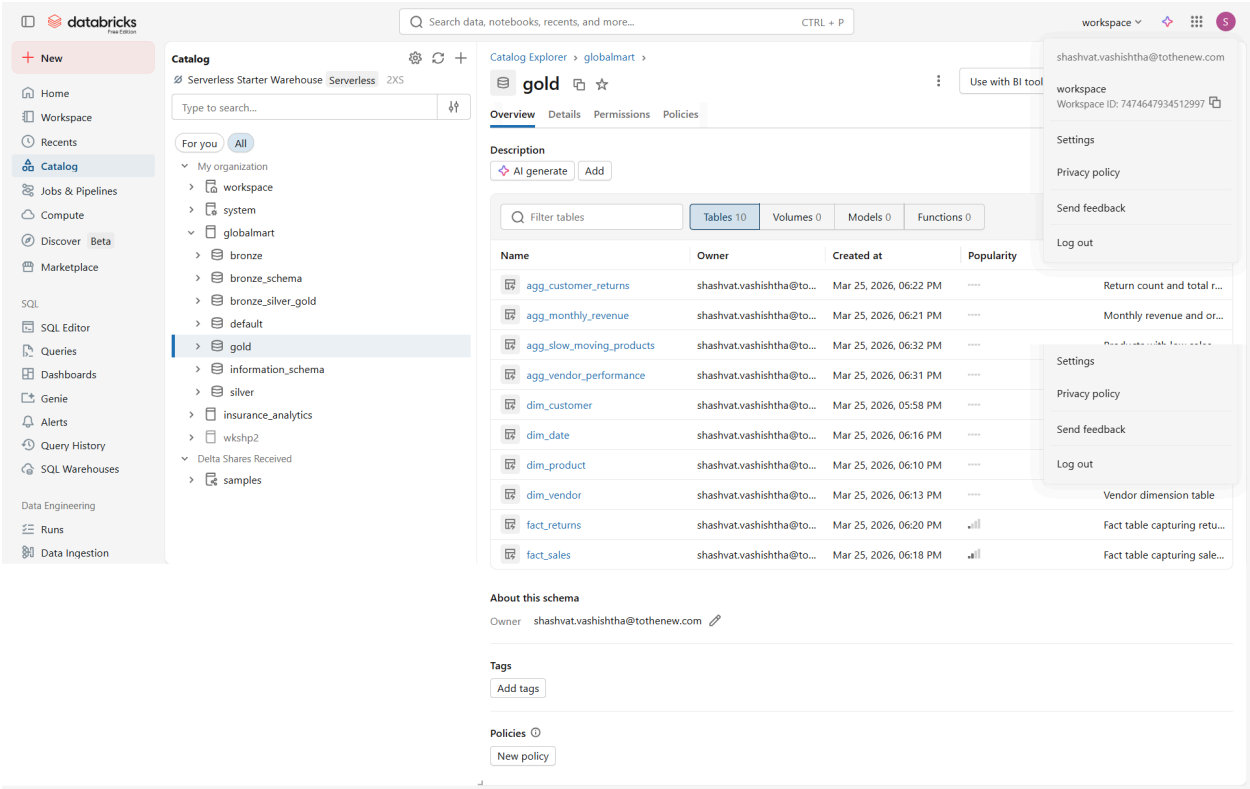
1.Screenshot of Unity Catalog showing the Gold schema with all fact tables, dimension tables, and Materialized Views created



2.Build Issues Encountered During the Pipeline & How They Were Resolved

While building the Bronze → Silver → Gold pipeline and dimensional model, several common but significant issues were encountered. Each issue is documented below along with the impact and the exact steps taken to resolve it.

1. Join Failures Due to Inconsistent Customer IDs

What Happened

When joining `fact_transactions` with `dim_customers`, a large number of rows did not match, causing incorrect revenue totals by customer segment.

Root Cause

Customer identifiers were not consistent across regions at the Bronze layer:

- **Region 1 used `customer_id`**
- **Region 3 used `cust_id`**
- **Region 5 used `CustomerID` Additionally, duplicate customer records existed across regions.**

How It Was Resolved

- **In the Silver layer, all customer ID variations were normalized into a single `customer_id` column.**
- **Invalid records missing `customer_id` were routed to `customers_quarantine`.**
- **A unified `customers_silver` table was created and used as the source for `dim_customers`.**

Outcome:
All joins between `fact_transactions` and `dim_customers` now succeed, and customer segmentation metrics are correct.

---

2. Schema Mismatches Between Regions (Pipeline Failures)

What Happened

Auto Loader ingestion into Bronze initially failed when some regions had missing or additional columns (especially in orders and transactions).

Root Cause

Different regional source systems produced files with:

- **Extra columns in some regions**
- **Missing columns in others**
- **Different casing and naming conventions**

How It Was Resolved

- **Auto Loader was configured with:**
  - **Schema inference**
  - **Schema evolution enabled**
- **Column normalization logic was added at the start of every Silver pipeline (`normalize_columns()`).**
- **All missing fields were explicitly handled in Silver using NULL-safe logic.**

Outcome:
The pipeline became resilient to schema drift and ran successfully across all regions.

---

3. DLT Graph Did Not Show Bronze Tables

What Happened

The DLT pipeline graph only showed Silver base views → clean/quarantine tables. Bronze tables were not visible in the DAG, causing initial confusion.

Root Cause

Bronze tables were created outside of DLT using Auto Loader.
DLT only visualizes tables and views that it manages internally.

How It Was Resolved

- **Confirmed that this is expected DLT behavior.**
- **Documented bronze tables as *external upstream dependencies*.**
- **Ensured Silver tables correctly referenced Bronze via `dlt.read_stream()`.**

Outcome:
Pipeline behavior was verified as correct, and documentation clarified the lineage semantics.

4. Join Failures in Gold Layer Due to Date Format Differences

What Happened

Queries grouping revenue by month failed or produced incorrect results.

Root Cause

Dates in Silver tables came from multiple formats:

- `MM/dd/yyyy`
- `yyyy-MM-dd`
- **JSON date strings**

How It Was Resolved

- **All transactional and return dates were normalized in Silver into proper DATE/TIMESTAMP fields.**
- **A dedicated `dim_dates` table was created.**
- **Fact tables now store `date_key` instead of raw date strings.**

Outcome:

Time-based analytics (monthly revenue, quarterly trends) now work correctly.

5. Incorrect Revenue Due to Duplicate Orders

What Happened

Revenue totals were higher than expected when aggregating transactions.

Root Cause

Duplicate order rows existed due to:

- **Multiple regional ingestion of the same customer orders**
- **No clear fact grain initially enforced**

How It Was Resolved

- **Defined a strict grain for `fact_transactions`:**
  one row = one order line item
- **Deduplicated transactions in Silver where needed.**
- **Ensured only clean Silver data feeds Gold facts.**

Outcome:
Revenue audit discrepancies were eliminated.

6. DLT Streaming Errors When Using `dlt.read()` Instead of `dlt.read_stream()`

What Happened

DLT threw errors when Silver tables attempted to read from streaming Bronze tables using `dlt.read()`.

Root Cause

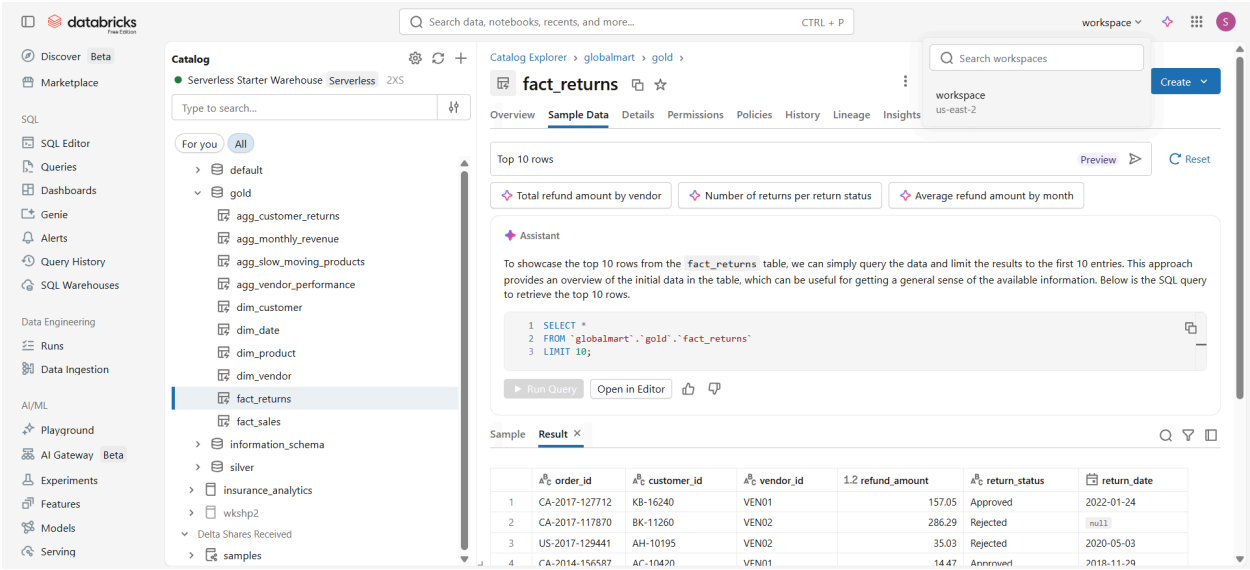
Bronze tables were created using streaming Auto Loader.
Downstream DLT tables must use `dlt.read_stream()` for compatibility.

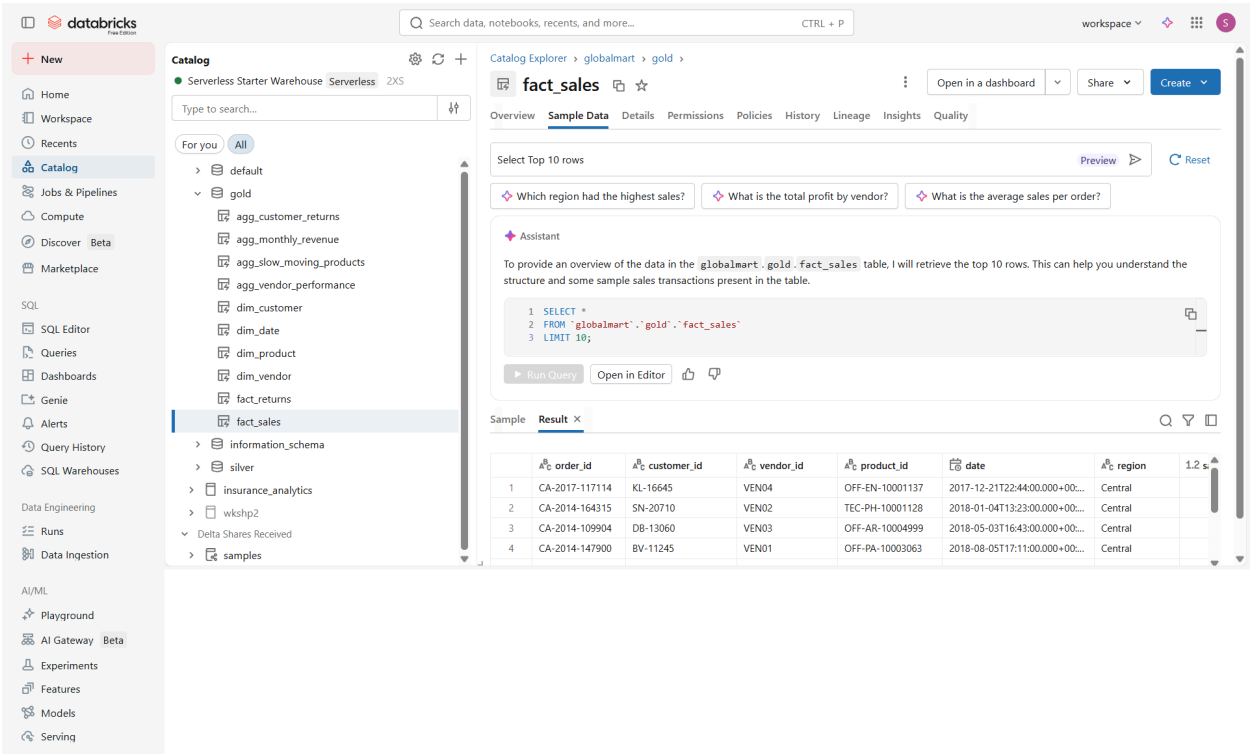
How It Was Resolved

- **Replaced all `dlt.read()` calls with `dlt.read_stream()` in Silver base views.**

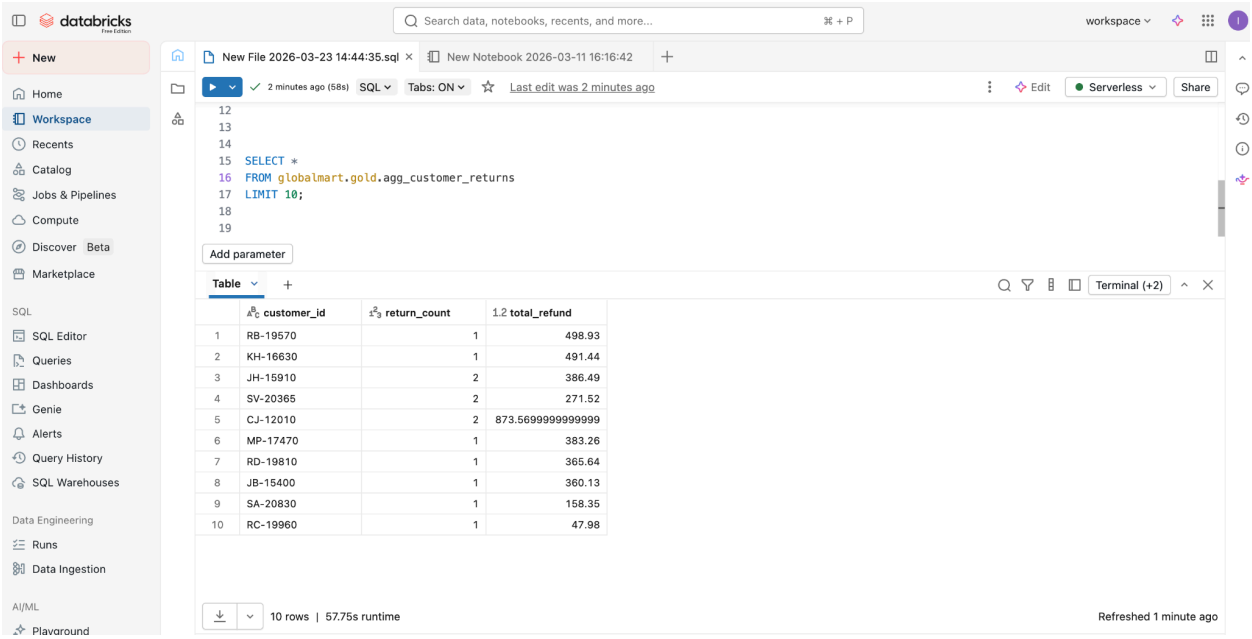
✅ **Outcome:**
Streaming pipeline stabilized and ran incrementally without issues.

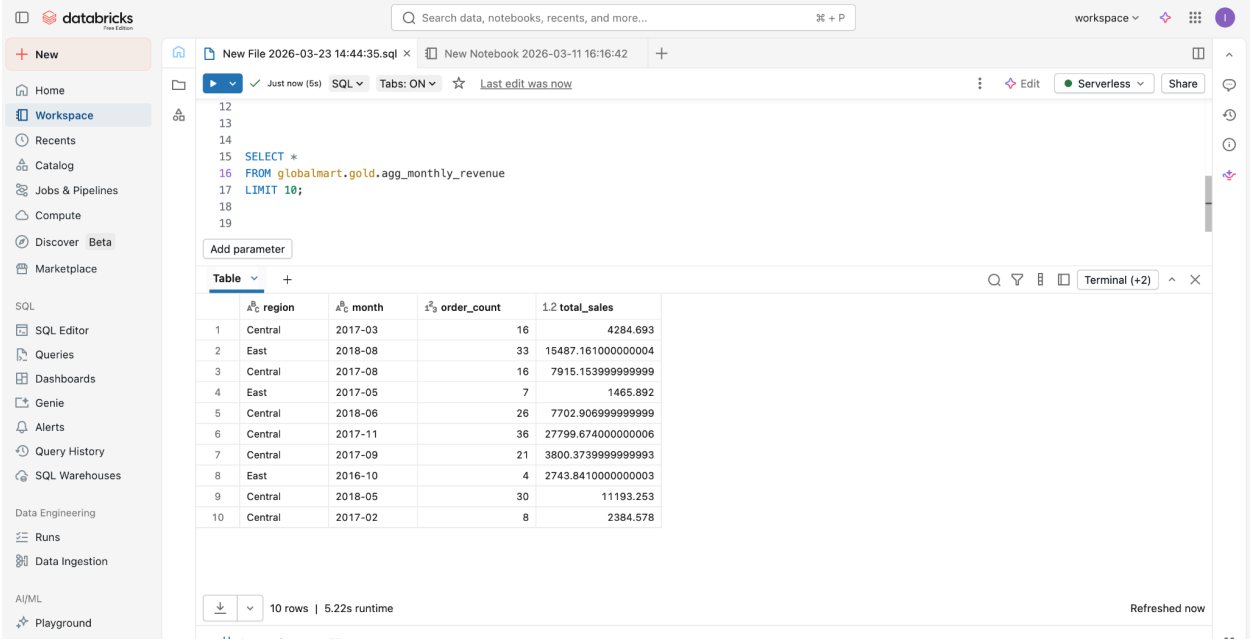
3. Screenshot of sample rows from your fact tables, results must be populated with actual data

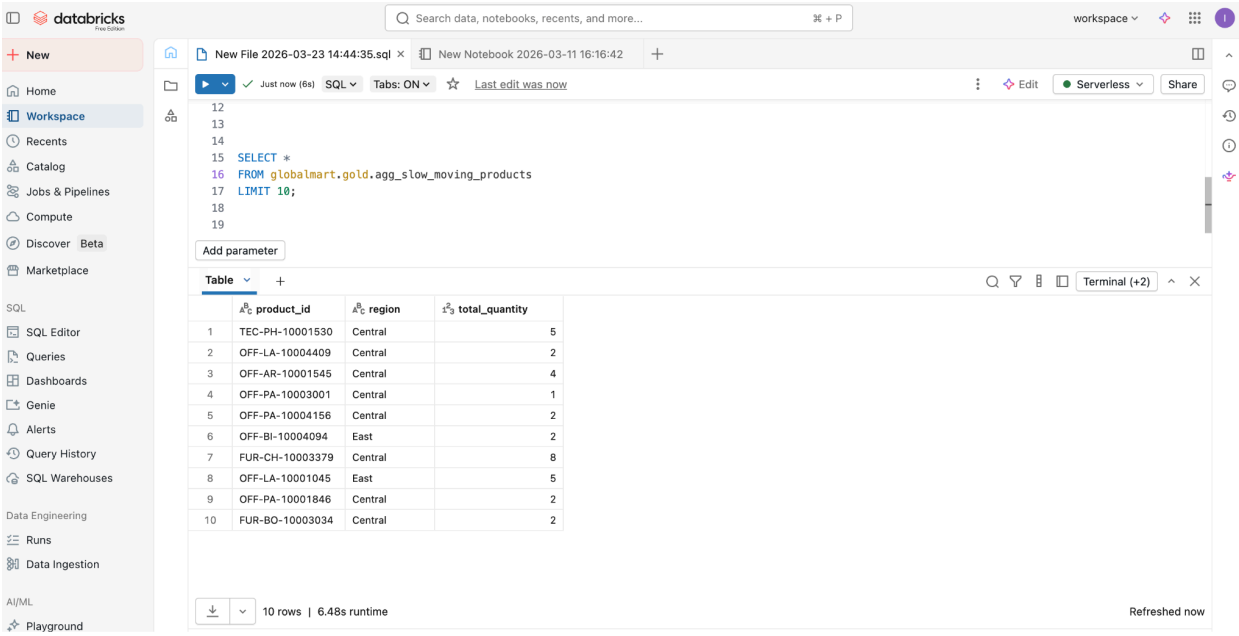




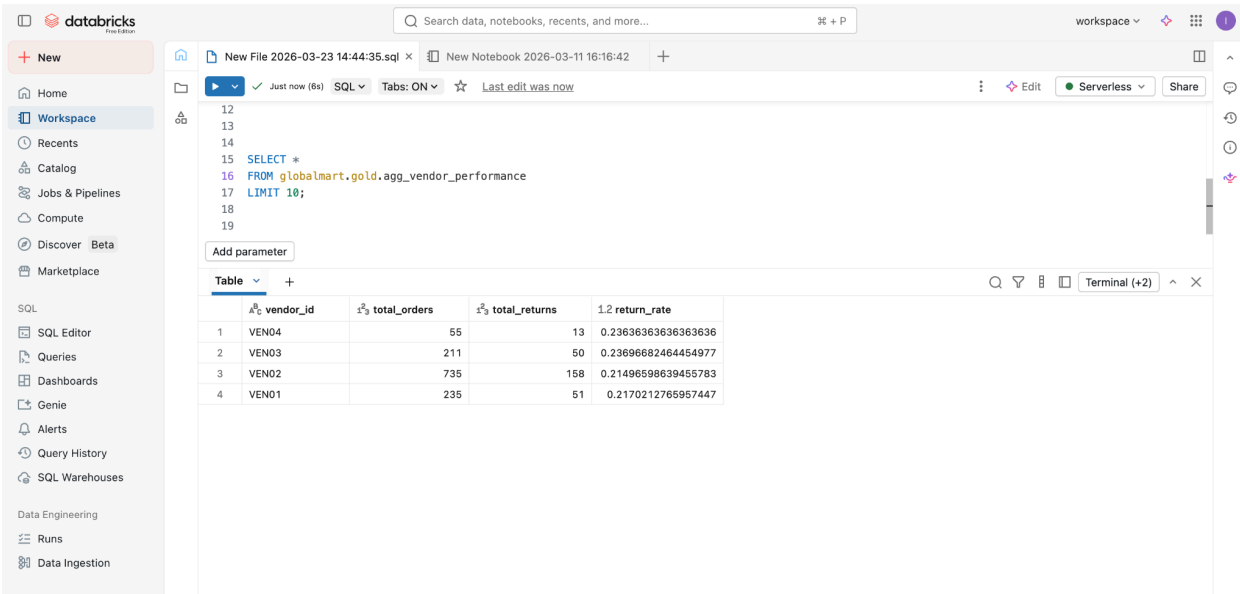
4.Screenshot of query results from each aggregation, results must be populated with actual data





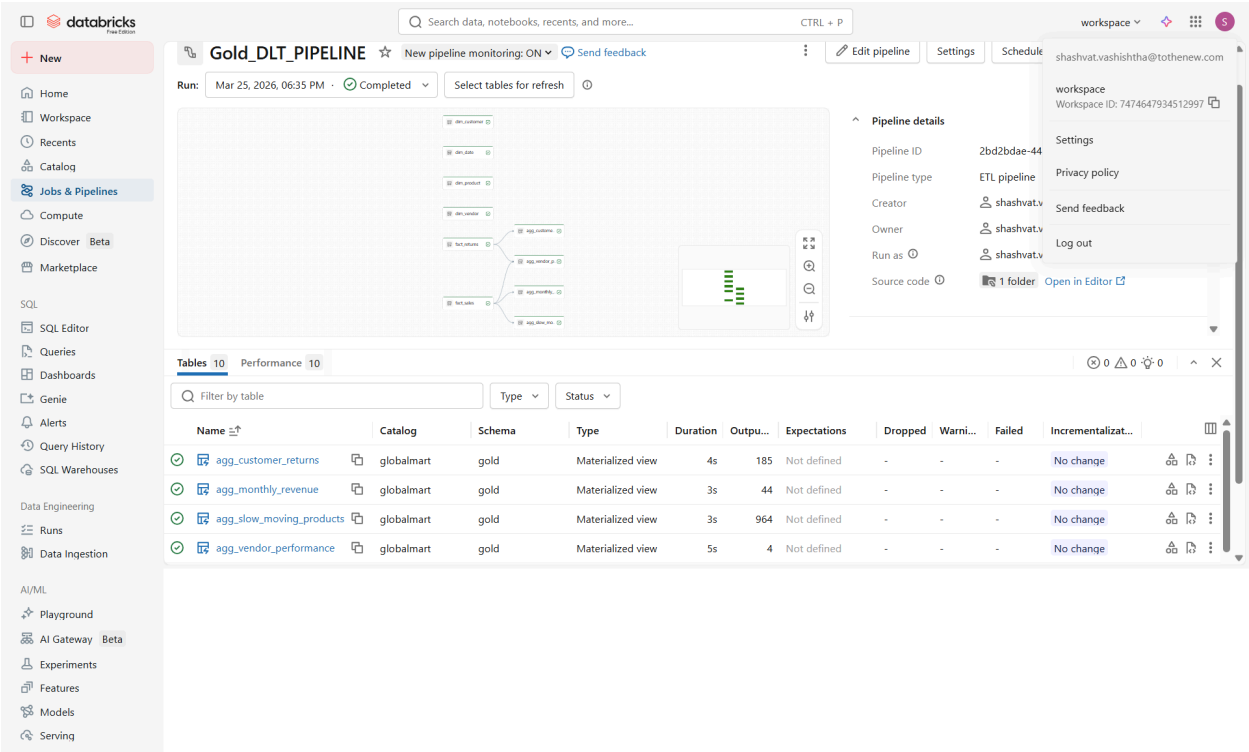


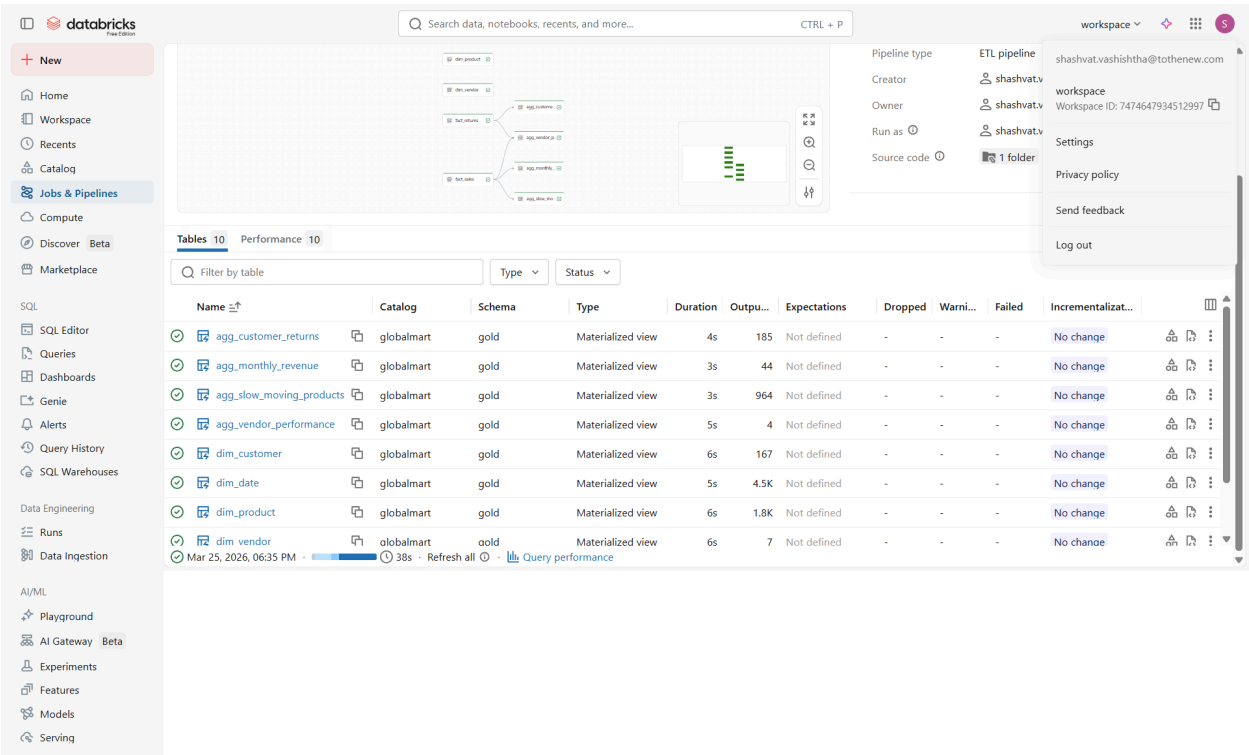
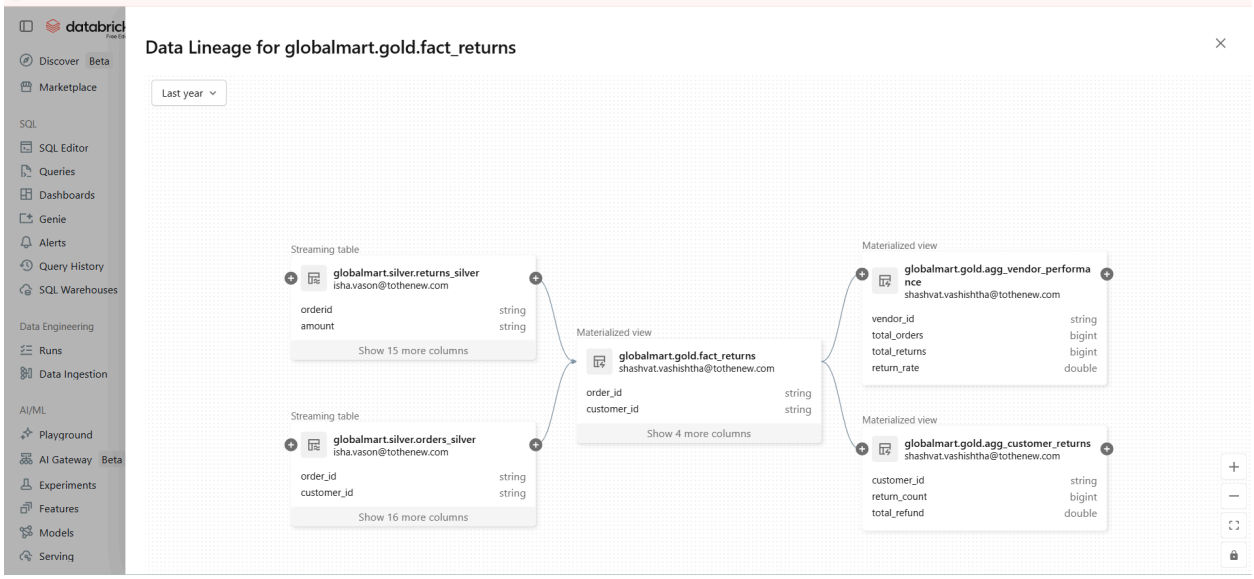
	product_id	region	total_quantity
1	TEC-PH-10001530	Central	5
2	OFF-LA-10004409	Central	2
3	OFF-AR-10001545	Central	4
4	OFF-PA-10003001	Central	1
5	OFF-PA-10004156	Central	2
6	OFF-BI-10004094	East	2
7	FUR-CH-10003379	Central	8
8	OFF-LA-10001045	East	5
9	OFF-PA-10001846	Central	2
10	FUR-BO-10003034	Central	2



	vendor_id	total_orders	total_returns	return_rate
1	VEN04	55	13	0.23636363636363636
2	VEN03	211	50	0.23696682464454977
3	VEN02	735	158	0.21496598639455783
4	VEN01	235	51	0.2170212765957447

5.Screenshot of a successful pipeline run that includes the Gold layer





Screenshot showing the lineage of Gold layer tables and aggregations included as part of the pipeline run